

Full-Stack E-Commerce Project

.NET + Angular + SignalR + Google Sign-In

A step-by-step Learn → Apply roadmap for building a professional, CV-ready project

What is inside

1. How to learn with this roadmap (the method)
2. Project overview, feature list, tech stack, architecture and timeline
3. Sixteen stages (0-15), each with: Learn → Apply → Deliverables → Checklist
4. **Google (Gmail) Sign-In**, full guide with Google Cloud setup and code (Stage 4)
5. **SignalR** real-time features (Stage 9)
6. CV bullets, interview questions, resources, golden rules, progress tracker

Contents

How to Use This Roadmap (the learning method)	3
1. Project Overview	3
2. Tech Stack	4
3. Architecture	4
4. Recommended Timeline	5
Stage 0: Prerequisites	5
Stage 1: Planning and Design	6
Stage 2: Backend Foundation	6
Stage 3: Catalog API	7
Stage 4: Authentication and Authorization (JWT + Google Sign-In)	8
Stage 5: Angular Foundation	11
Stage 6: Storefront (Angular)	11
Stage 7: Basket (Redis + Angular State)	12
Stage 8: Checkout, Orders and Payments	12
Stage 9: SignalR (Real-Time Features) ★	13
Stage 10: Admin Dashboard	14
Stage 11: Advanced Backend Features	14
Stage 12: Testing	15
Stage 13: Security and Performance	15
Stage 14: DevOps, CI/CD and Deployment	16
Stage 15: Polish and CV Packaging	16
Recommended Learning Resources	18
Golden Rules While Building	18
Master Progress Tracker	18

How to Use This Roadmap (the learning method)

Every stage follows the same loop:

1. LEARN the topics of the stage → **2. APPLY** them in the project → **3. VERIFY** with the checklist → **4. COMMIT** to GitHub and move on.

The learn → apply loop is one of the most effective ways to learn programming, because each topic is attached to something real. To get the best result, follow these rules:

- **Build an MVP first (Stages 0-8):** catalog, login (including Google), basket, checkout and orders working end to end. That alone is already a good project.
- **Then upgrade it (Stages 9-15):** SignalR, admin dashboard, caching, tests, Docker, CI/CD. Each one improves something that already works, which is easier to learn.
- **Build vertical slices:** take one feature all the way (database → API → Angular screen) before starting the next one. You see results faster and stay motivated.
- **Learn just enough, then build.** Do not study a whole stage for a week before writing code, you will forget half of it. Learn one small piece, build it now, then learn the next.
- **Understand every line.** After each feature ask: could I rebuild this without looking, and explain why it works? If not, redo it.
- **Keep a short 'What I learned' note** after each stage. It becomes your interview cheat sheet.
- **Trim if you are a beginner:** postpone CQRS/MediatR, RabbitMQ, Elasticsearch and the Outbox pattern. A finished, deployed project with clean code beats an unfinished 'perfect' one.

1. Project Overview

Project name (suggestion): Oranos Shop (or any brand name you like).

A complete online store: a customer storefront (Angular), an admin dashboard, and a secure REST API with a real-time layer (.NET).

Customer side

- Register / login with email + password (JWT + refresh token), email confirmation, forgot password
- **Sign in with Google (Gmail account)** in one click
- Browse products with search, filters (brand, category, price), sorting and pagination
- Product details, images, reviews and ratings; wishlist
- Shopping basket stored in Redis (works before login)
- Checkout: address, delivery method, online payment; order history and tracking
- **Real-time notifications (SignalR):** order status changes, stock alerts, flash-sale banners
- Arabic / English (i18n) with RTL support

Admin side

- Dashboard with charts (sales, orders, top products)
- Product / category / brand CRUD with image upload
- Order management (changing a status pushes a live notification to the customer via SignalR)
- Users and roles management
- **Live admin feed (SignalR):** a new order appears instantly

Non-functional (this is what makes the CV strong)

- Clean Architecture, SOLID and design patterns
- Automated tests, CI/CD pipeline, Docker
- Logging, health checks, caching, rate limiting, security best practices
- Deployed live with a public URL

2. Tech Stack

Layer	Technology
Backend	ASP.NET Core Web API (latest .NET LTS), C#
ORM / Database	Entity Framework Core + SQL Server (or PostgreSQL)
Cache / Basket	Redis
Authentication	ASP.NET Core Identity + JWT + Refresh Tokens + Google Sign-In (Google.Apis.Auth)
Real-time	SignalR (Redis backplane for scaling)
Validation / Mapping	FluentValidation, Mapster or AutoMapper
Patterns	Repository, Unit of Work, Specification, CQRS + MediatR (optional but strong)
Background jobs	Hangfire (emails, cleanup, reports)
Logging	Serilog (+ Seq or Application Insights)
Payments	Stripe (international) or Paymob (Egypt)
Frontend	Angular (latest), TypeScript, RxJS, Signals
UI	Angular Material or Tailwind CSS
State	Signals + services (or NgRx for the basket)
Testing	xUnit, Moq/NSubstitute, FluentAssertions, Testcontainers, Jest/Karma, Playwright
DevOps	Docker, docker-compose, GitHub Actions, Azure / Render / Railway

3. Architecture

Backend: Clean Architecture

```
src/  
  Shop.Domain           Entities, Enums, Domain Exceptions, Interfaces (no dependencies)  
  Shop.Application      Use cases (CQRS), DTOs, Validators, Interfaces, Mapping  
  Shop.Infrastructure    EF Core, Repositories, Redis, Email, Payment, Google auth, Identity  
  Shop.API              Controllers, Middleware, SignalR Hubs, DI setup, Program.cs  
tests/  
  Shop.UnitTests  
  Shop.IntegrationTests
```

Dependency rule: API → Application → Domain and Infrastructure → Application → Domain. The Domain project depends on nothing.

Frontend: feature-based structure

```
src/app/  
  core/      interceptors, guards, auth service (incl. Google), signalr service, layout  
  shared/    reusable components, pipes, directives, models  
  features/  
    shop/    product list, product details  
    basket/  
    checkout/  
    account/ login (email + Google), register, profile, orders  
    admin/   dashboard, products, orders, users  
  app.routes.ts lazy-loaded routes
```

Main database entities

Product, ProductImage, Category, Brand, Review, Wishlist, Basket and BasketItem (Redis), Order, OrderItem, DeliveryMethod, Address, Payment, Coupon, Notification, AppUser, AppRole, and Identity's external-login table (used for Google accounts).

4. Recommended Timeline

Stage	Topic	Estimated time
0	Prerequisites	1-2 weeks (skip what you know)
1	Planning and Design	3-4 days
2	Backend Foundation	1 week
3	Catalog API	1-1.5 weeks
4	Authentication (JWT + Google Sign-In)	1-1.5 weeks
5	Angular Foundation	1 week
6	Storefront (Angular)	1.5 weeks
7	Basket (Redis + Angular)	1 week
8	Checkout, Orders, Payments	1.5 weeks
9	SignalR (Real-time)	1 week
10	Admin Dashboard	1.5 weeks
11	Advanced Backend Features	1 week
12	Testing	1 week
13	Security and Performance	3-4 days
14	DevOps, CI/CD, Deployment	1 week
15	Polish and CV Packaging	3-4 days

Total: roughly **15-19 weeks** at a steady part-time pace. It is better to finish everything at 80% quality than to leave stages half-done.

Stage 0: Prerequisites

■ LEARN

- **C#**: OOP, interfaces, generics, LINQ, async/await, delegates, extension methods, nullable reference types, records

- **SQL:** SELECT, JOIN, GROUP BY, indexes, transactions, normalization, relationships (1-1, 1-M, M-M)
- **Web basics:** HTTP methods, status codes, headers, REST principles, JSON, CORS
- **Git and GitHub:** branches, commits, pull requests, `.gitignore`, conventional commits
- **Frontend basics:** HTML5, CSS3 (Flexbox, Grid), modern JavaScript (ES6+) and **TypeScript**

■ APPLY (BUILD IT)

1. Install: .NET SDK, Visual Studio or VS Code / Rider, SQL Server (or the Docker image), Node.js LTS, Angular CLI, Git, Postman or Bruno, Docker Desktop
2. Create a GitHub repository (monorepo with `/backend` and `/frontend` folders)
3. Build two tiny practice apps: a console app using LINQ, and a static page using TypeScript

■ CHECKLIST (finish before moving on)

- ☐ I can explain async/await and LINQ deferred execution
- ☐ I can write a JOIN and a GROUP BY query without help
- ☐ Repository created and first commit pushed

Stage 1: Planning and Design

■ LEARN

- Requirements gathering: functional vs non-functional
- User stories and acceptance criteria
- ERD (Entity Relationship Diagram) design
- REST API design: naming, versioning, status codes, error format (ProblemDetails)
- Wireframing (Figma free plan)

■ APPLY (BUILD IT)

1. Write the feature list as user stories (GitHub Issues or Projects board)
2. Draw the ERD (draw.io or dbdiagram.io)
3. Draw the architecture diagram (it goes in your README later)
4. Design the API endpoint list before coding
5. Make simple wireframes: home, product list, product details, basket, checkout, login, admin dashboard

■ DELIVERABLES

- A `docs/` folder with the ERD, architecture diagram, wireframes and API endpoint list

■ CHECKLIST (finish before moving on)

- ☐ ERD covers all entities and relationships
- ☐ GitHub board has issues for each stage

Stage 2: Backend Foundation

■ LEARN

- ASP.NET Core basics: Program.cs, middleware pipeline, dependency injection (Singleton / Scoped / Transient)
- Configuration: `appsettings.json`, environments, Options pattern, User Secrets

- **Entity Framework Core:** DbContext, Fluent API configuration, migrations, relationships, `Include`, tracking vs no-tracking
- Clean Architecture and the dependency rule
- Data seeding

■ APPLY (BUILD IT)

1. Create the solution with 4 projects (Domain, Application, Infrastructure, API)
2. Add entities: Product, Category, Brand, ProductImage
3. Configure entities with `IEntityTypeConfiguration<T>` classes
4. Create `StoreDbContext`, the first migration, and apply it to the database
5. Seed categories, brands and about 50 realistic products (JSON seed files)
6. Register services with extension methods (`AddInfrastructure()`, `AddApplication()`)

■ CHECKLIST (finish before moving on)

- ☐ Database is created by migrations and seeded with data
- ☐ The Domain project has zero references to other projects
- ☐ Connection string is in User Secrets, not in Git

Stage 3: Catalog API

■ LEARN

- RESTful controllers vs Minimal APIs
- DTOs, and why you never return entities directly
- **Generic Repository + Unit of Work**, and when they are (and are not) useful with EF Core
- **Specification pattern** (filtering, sorting, paging, includes in one reusable object)
- Pagination (offset vs cursor), sorting, searching
- Mapping with Mapster or AutoMapper
- **FluentValidation**
- **Global exception handling** middleware and `ProblemDetails`
- Swagger / OpenAPI
- Strong add-on: **CQRS with MediatR** and pipeline behaviors (validation, logging)

■ APPLY (BUILD IT)

1. Build `GET /api/products` with filters (brand, category, price range), search, sorting and pagination, returning `{ pageIndex, pageSize, count, data }`
2. Add `GET /api/products/{id}`, `GET /api/brands`, `GET /api/categories`
3. Implement the Specification pattern for product queries
4. Add a global exception middleware that returns a consistent error JSON
5. Add FluentValidation for query parameters
6. Configure Swagger with XML comments and JWT support (needed later)
7. Add API versioning (`/api/v1/...`)

■ CHECKLIST (finish before moving on)

- ☐ Every response uses DTOs
- ☐ Errors always come back in the same format
- ☐ Swagger documents all endpoints

- ❑ I can explain why the Specification pattern beats writing LINQ inside controllers

Stage 4: Authentication and Authorization (JWT + Google Sign-In)

4.1 Email/password authentication

■ LEARN

- Authentication vs Authorization
- **ASP.NET Core Identity** (users, roles, password hashing, logout)
- **JWT**: structure, claims, signing, expiry, validation
- **Refresh tokens**: rotation, revocation, secure storage (HttpOnly cookie vs local storage trade-offs)
- Role-based and policy-based authorization
- Email confirmation and password reset flows

■ APPLY (BUILD IT)

1. Add Identity (separate `AppIdentityDbContext` or a shared context)
2. Endpoints: `register`, `login`, `refresh-token`, `logout`, `confirm-email`, `forgot-password`, `reset-password`, `GET/PUT me`, address CRUD
3. Seed roles `Admin` and `Customer`, and one admin user
4. Protect endpoints with `[Authorize]` and `[Authorize(Roles = "Admin")]`
5. Send emails through an `IService` (MailKit or SendGrid; Mailtrap in development)
6. Add login attempt limiting (logout)

4.2 Sign in with Google (Gmail account) ★

This lets a user sign in with one click using their Google account. It also looks great on a CV because it shows you understand **OAuth 2.0 / OpenID Connect** and token validation.

How it works (the flow we will build)

1. The Angular app shows the official **Sign in with Google** button (Google Identity Services).
2. The user picks a Gmail account in the Google popup. Google returns a signed **ID token** (a JWT) to Angular.
3. Angular sends this ID token to your API: `POST /api/auth/google` with `{ "idToken": "...", ... }`.
4. The API **validates the token with Google's libraries** (signature, expiry, and that the audience is your Client ID).
5. The API finds the user (by Google id, then by email) or **creates** a new one, and links the Google login.
6. The API returns **your own** access token + refresh token, exactly like a normal login. From here on, the user is a normal user: roles, basket, orders and SignalR all work the same way.

Important: never trust the email or name sent by the browser. Always validate the ID token on the server. Never send or store a Google Client Secret in Angular. This flow needs only the Client ID.

Topics to learn first

- OAuth 2.0 vs OpenID Connect: authentication (who are you) vs authorization (what can you access)

- What an ID token is, its claims (`sub`, `email`, `email_verified`, `aud`, `iss`, `exp`) and how it is signed
- Google Identity Services (GIS) for the web
- Account linking: what to do when the email already exists as a normal account
- ASP.NET Core Identity external logins (`UserLoginInfo`, `AddLoginAsync`, `FindByLoginAsync`)

Step A: Google Cloud Console setup

1. Open Google Cloud Console and create a new project (for example `oranos-shop`).
2. Go to **APIs & Services** → **OAuth consent screen**. Choose **External**, fill the app name, support email and developer email. While the app is in Testing mode, add your own Gmail as a **test user**.
3. Go to **APIs & Services** → **Credentials** → **Create credentials** → **OAuth client ID**, and choose application type **Web application**.
4. Under **Authorized JavaScript origins** add `http://localhost:4200` (development) and later your production URL (for example `https://shop.yourdomain.com`).
5. Copy the **Client ID** (it ends with `.apps.googleusercontent.com`). You do not need the client secret for this flow.
6. Before going live, publish the consent screen (move from Testing to In production) so any Gmail user can sign in.

Menu names in Google Cloud Console change from time to time, so if a label looks different, use the official Google Identity documentation as a guide.

Step B: Backend (ASP.NET Core)

Install the NuGet package `Google.Apis.Auth` in the Infrastructure project, and store the Client ID in configuration (`Google:ClientId`), not in code.

```
public async Task<AuthResponse> GoogleLoginAsync(string idToken)
{
    var settings = new GoogleJsonWebSignature.ValidationSettings
    {
        Audience = new[] { _config["Google:ClientId"] } // must be YOUR client id
    };

    // Throws InvalidJwtException if signature / audience / expiry is wrong
    var payload = await GoogleJsonWebSignature.ValidateAsync(idToken, settings);

    if (!payload.EmailVerified)
        throw new UnauthorizedException("Google email is not verified");

    // 1) Already linked to Google?
    var user = await _userManager.FindByLoginAsync("Google", payload.Subject);

    if (user is null)
    {
        // 2) Same email registered normally? Link it. Otherwise create a new user.
        user = await _userManager.FindByEmailAsync(payload.Email);
        if (user is null)
        {
            user = new AppUser
            {
                UserName = payload.Email, Email = payload.Email,
                EmailConfirmed = true, DisplayName = payload.Name
            };
            await _userManager.CreateAsync(user);
            await _userManager.AddToRoleAsync(user, "Customer");
        }
        await _userManager.AddLoginAsync(user,
            new UserLoginInfo("Google", payload.Subject, "Google"));
    }

    return await _tokenService.CreateTokensAsync(user); // your own JWT + refresh token
}

// AuthController
[HttpPost("google")]
```

```
public async Task<ActionResult<AuthResponse>> Google(GoogleLoginDto dto)
    => Ok(await _authService.GoogleLoginAsync(dto.IdToken));
```

Step C: Frontend (Angular)

Add Google's script to `index.html`, then render the button and send the returned credential to your API. (Alternative: the community library [@abacritt/angularx-social-login](#) wraps this for you, but learn the manual way first.)

```
<!-- index.html -->
<script src="https://accounts.google.com/gsi/client" async defer></script>

// login.component.ts
declare const google: any;

ngAfterViewInit() {
  google.accounts.id.initialize({
    client_id: environment.googleClientId,
    callback: (resp: any) =>
      this.zone.run(() =>
        this.auth.googleLogin(resp.credential).subscribe(() => this.router.navigate(['/']))
      )
  });
  google.accounts.id.renderButton(
    document.getElementById('googleBtn'),
    { theme: 'outline', size: 'large', text: 'signin_with' }
  );
}

// auth.service.ts
googleLogin(idToken: string) {
  return this.http.post<AuthResponse>(`${this.api}/auth/google`, { idToken });
}
```

Add `<div id="googleBtn"></div>` to the login and register pages, under the normal form, with an "or" separator. Store the returned tokens the same way as in a normal login.

Edge cases to handle (this is what makes it professional)

- **Account linking:** an existing email/password user signs in with Google using the same email. Link the accounts only because Google says `email_verified = true`.
- **New users:** create the account automatically with role `Customer` and `EmailConfirmed = true`.
- **Google-only users have no password:** if they use 'forgot password', let them set one, or show a clear message.
- **Profile picture and name:** save them from the token claims on first login.
- **Errors:** invalid or expired token returns 401 with a `ProblemDetails` body; show a friendly toast in Angular.
- **Rate limiting and logging:** apply the same rate limit as `login`, and log failed validations (without logging tokens).
- **Environments:** use separate Client IDs for development and production.

■ CHECKLIST (finish before moving on)

- ☐ A new Gmail account signs in and gets a Customer account created automatically
- ☐ The same Gmail signing in twice does not create a duplicate user
- ☐ A fake or expired ID token is rejected with 401
- ☐ An ID token issued for a different Client ID is rejected (audience check)
- ☐ After Google login the user can use the basket, checkout and SignalR normally
- ☐ Access token expires in minutes; the refresh token works and rotates
- ☐ Admin endpoints return 403 for normal users; password rules and lockout are tested

Stage 5: Angular Foundation

■ LEARN

- Angular architecture: standalone components, templates, data binding, control flow (`@if`, `@for`)
- **Signals** (`signal`, `computed`, `effect`) plus RxJS basics (`Observable`, `map`, `switchMap`, `catchError`, `debounceTime`)
- Routing: lazy loading, route guards, resolvers
- Services and dependency injection (`inject()`)
- `HttpClient` and **interceptors** (JWT, error handling, loading spinner)
- Template-driven vs **Reactive Forms**, custom validators
- UI library: Angular Material or Tailwind
- Environment configuration

■ APPLY (BUILD IT)

1. Create the Angular app and set up the core / shared / features folders
2. Build the layout: navbar, footer, sidebar, toast notifications, loading indicator
3. Create `AuthService` and interceptors: `jwtInterceptor`, `errorInterceptor` (401 → refresh token or redirect), `loadingInterceptor`
4. Create guards: `authGuard`, `adminGuard`
5. Configure lazy-loaded routes for each feature
6. Generate typed models from your Swagger (NSwag or OpenAPI Generator)

■ CHECKLIST (finish before moving on)

- ☐ Features load lazily (verify in the browser network tab)
- ☐ 401 responses are handled automatically
- ☐ Environment files separate the dev and prod API URLs and Google Client ID

Stage 6: Storefront (Angular)

■ LEARN

- Component communication (`input()`, `output()`, services)
- Pagination, search debounce and filter UI patterns
- Image lazy loading (`NgOptimizedImage`)
- Change detection strategy `OnPush`
- SEO basics; bonus: **Angular SSR**
- Responsive design

■ APPLY (BUILD IT)

1. **Home page:** hero, featured products, categories
2. **Product list:** search, filters, sorting, pagination (connected to the Stage 3 API)
3. **Product details:** image gallery, related products, reviews, add-to-basket button
4. **Account pages:** login (email form + **Google button**), register, forgot password, profile, addresses
5. Skeleton loaders, empty states and error states
6. Add Arabic/English with `@angular/localize` or `ngx-translate`, and an RTL layout

■ CHECKLIST (finish before moving on)

- ☐ Filters and pagination are reflected in the URL query string
- ☐ Works well on mobile
- ☐ Lighthouse performance score is 85+

Stage 7: Basket (Redis + Angular State)

■ LEARN

- Redis basics: key-value, TTL, data types, `StackExchange.Redis`
- Distributed caching in .NET
- State management: a service with Signals, or **NgRx** (store, actions, reducers, effects, selectors)
- Optimistic UI updates

■ APPLY (BUILD IT)

1. Run Redis with Docker
2. Backend: `IBasketRepository` using Redis; endpoints `GET/POST/DELETE /api/basket`; TTL of 30 days
3. Frontend: `BasketService` with signals (`items`, computed `totals`); keep the basket id in `localStorage`
4. Basket page: update quantity, remove item, coupon code, order summary
5. Merge the anonymous basket into the user's basket after login (email or Google)

■ CHECKLIST (finish before moving on)

- ☐ The basket survives a page refresh and works before login
- ☐ Totals are also calculated on the **server** (never trust client prices)

Stage 8: Checkout, Orders and Payments

■ LEARN

- Multi-step checkout UX (Angular Material Stepper)
- Domain modeling: Order aggregate, Value Objects, order status state machine
- **Database transactions**, concurrency (`RowVersion`), inventory handling
- **Payments**: Stripe PaymentIntents (or Paymob), **webhooks**, idempotency
- Status flow: `Pending` → `PaymentReceived` → `Processing` → `Shipped` → `Delivered` / `Cancelled` / `PaymentFailed`

■ APPLY (BUILD IT)

1. Entities: `Order`, `OrderItem` (snapshot of product name/price at purchase time), `DeliveryMethod`, `Address`
2. `POST /api/orders` reads the basket from Redis and **recalculates prices from the database**
3. Create and update the payment intent when the basket changes
4. Webhook endpoint that confirms payment and updates the order status (verify the signature!)
5. Angular checkout: Address → Delivery → Review → Payment → Success page
6. Order history page and order details page
7. Reduce stock on successful payment (inside a transaction, with concurrency handling)
8. Send the order confirmation email through a background job

■ CHECKLIST (finish before moving on)

- ☐ Changing prices in the browser cannot change what the user pays
- ☐ The payment webhook is idempotent (the same event twice = no double processing)
- ☐ Two users buying the last item at the same time is handled

Stage 9: SignalR (Real-Time Features) ★

This is the stage that sets your project apart from typical CRUD projects.

■ LEARN

- What SignalR is: WebSockets, Server-Sent Events, Long Polling, and how SignalR chooses between them
- Hubs, clients, groups, connection lifecycle (`OnConnectedAsync`, `OnDisconnectedAsync`)
- Strongly-typed hubs (`Hub<IClient>`)
- **Authenticating SignalR with JWT** (access token in the query string for WebSockets)
- Sending from outside a hub using `IHubContext<T>`
- Scaling SignalR: sticky sessions and the **Redis backplane**
- Angular client: `@microsoft/signalr`, wrapped in a service with automatic reconnect

Real-time features to build (pick at least the first three)

1. **Order status notifications:** when the admin changes an order status, the customer sees a live toast and the notification badge increases, without refreshing
2. **Live admin feed:** a new order appears instantly on the admin dashboard, with live counters for orders and revenue
3. **Stock / price updates:** live warning if an item in your basket runs low or its price changes
4. **Flash-sale banner:** the admin starts a sale and all online visitors see the banner immediately
5. **Live customer support chat** (customer ↔ admin) with typing indicator and history saved in the database
6. **Online users counter** on the admin dashboard

■ APPLY (BUILD IT)

1. Create `NotificationHub` and map it: `app.MapHub<NotificationHub>("/hubs/notifications")`
2. Authenticate connections with the JWT (this works for Google-login users too, because they get your own JWT); add each user to a group named by user id, and admins to an `admins` group
3. Store notifications in the database (`Notification` entity) so unread ones appear after reconnect
4. Inject `IHubContext<NotificationHub, INotificationClient>` into the order service to push events
5. Angular: create `SignalRService` (start/stop, `onreconnected`, expose events as signals or observables)
6. Add the Redis backplane for multiple server instances

■ CHECKLIST (finish before moving on)

- ☐ Two browsers open (customer and admin): changing the order status updates the customer instantly
- ☐ The connection reconnects automatically after a network drop
- ☐ Unauthorized users cannot join the admin group

Stage 10: Admin Dashboard

■ LEARN

- Data tables with server-side pagination and sorting
- File upload: `IFormFile`, validation (size/type), storage abstraction (local → Azure Blob / Cloudinary)
- Charts: ngx-charts or Chart.js
- Reactive forms with dynamic fields (image arrays, variants)
- Aggregation queries in EF Core (`GroupBy`, projections)

■ APPLY (BUILD IT)

1. **Dashboard:** total sales, orders today, new users, low-stock alerts, revenue chart (30 days), top 5 products (live-updated with SignalR)
2. **Products management:** create/edit/delete, multi-image upload, soft delete
3. **Categories and brands** management
4. **Orders management:** filters by status/date, change status, view details
5. **Users management:** list, lock/unlock, change roles
6. Export orders to Excel/CSV (ClosedXML or CsvHelper)
7. Audit fields on entities: `CreatedAt`, `UpdatedAt`, `CreatedBy` (in a `SaveChanges` override)

■ CHECKLIST (finish before moving on)

- ☐ Only the Admin role can open `/admin` (in the Angular guard **and** in the API)
- ☐ Image upload rejects wrong file types and huge files

Stage 11: Advanced Backend Features

■ LEARN

- Caching: in-memory, distributed (Redis), response caching, cache invalidation strategies
- **Rate limiting** middleware (built into ASP.NET Core)
- Background jobs with **Hangfire** (or `BackgroundService`)
- **Structured logging with Serilog**, correlation ids
- Health checks (`/health`)
- EF Core performance: `AsNoTracking`, projections, split queries, indexes, the N+1 problem
- Optional: domain events, Outbox pattern, **RabbitMQ** or Azure Service Bus, Elasticsearch / full-text search

■ APPLY (BUILD IT)

1. Cache the product list and brands/categories in Redis, and invalidate on updates
2. Add rate limiting: stricter for `login`, `register` and `auth/google`
3. Hangfire jobs: send emails, clear abandoned baskets, daily sales report, release stock of unpaid orders after 30 minutes
4. Serilog to console + file (+ Seq container), logging request duration and user id
5. Health checks for the database and Redis
6. Find and fix N+1 queries using EF logging, add database indexes, and record before/after measurements for the README

■ CHECKLIST (finish before moving on)

- ☐ Cache hit vs miss is visible in the logs

- ☐ Failed emails retry automatically
- ☐ I can show a measured performance improvement

Stage 12: Testing

■ LEARN

- The test pyramid: unit → integration → end-to-end
- **xUnit**, Moq or NSubstitute, FluentAssertions, AutoFixture
- Integration tests with **WebApplicationFactory** and **Testcontainers** (real SQL Server / Redis in Docker)
- Angular testing: component tests (Jest or Karma + Jasmine), service tests with **HttpTestingController**
- **Playwright** end-to-end tests
- Code coverage (Coverlet)

■ APPLY (BUILD IT)

1. Unit tests: pricing/total calculation, order state transitions, validators, handlers (aim for 70%+ on the Application layer)
2. Unit test the Google login service by mocking the token validator (new user, existing user, unverified email)
3. Integration tests: register → login → add to basket → create order
4. Angular: test **BasketService**, the auth guard and the product list component
5. Playwright: one main E2E scenario (search → add to cart → checkout with a test card)
6. Show the test coverage badge in the README

■ CHECKLIST (finish before moving on)

- ☐ **dotnet test** passes on a clean machine
- ☐ At least one full E2E test runs in CI

Stage 13: Security and Performance

■ LEARN

- **OWASP Top 10**: injection, broken authentication, XSS, CSRF, insecure deserialization, misconfiguration
- CORS done correctly, HTTPS/HSTS, security headers (CSP, X-Content-Type-Options)
- Secrets management (User Secrets, environment variables, Azure Key Vault)
- Input validation and output encoding
- Angular's built-in XSS protection and **DomSanitizer**
- Performance: response compression, HTTP/2, bundle size analysis, lazy loading, image optimization

■ APPLY (BUILD IT)

1. Write a security review against the OWASP Top 10 and document it
2. Add a security-headers middleware; restrict CORS to your frontend origin only
3. Remove all secrets from Git history; use environment variables
4. Confirm the Google login validates the **audience** and rejects unverified emails

5. Run `npm audit` and `dotnet list package --vulnerable`
6. Check Lighthouse and the bundle analyzer; fix the biggest issues

■ CHECKLIST (finish before moving on)

- ☐ No secrets in the repository
- ☐ Security headers score A on securityheaders.com after deployment

Stage 14: DevOps, CI/CD and Deployment

■ LEARN

- **Docker:** Dockerfile (multi-stage), images vs containers, volumes, networks
- **docker-compose:** API + Angular (nginx) + SQL Server + Redis + Seq in one command
- **GitHub Actions:** build → test → build image → deploy
- Environments: Development / Staging / Production
- Cloud hosting options: Azure App Service + Azure SQL, or free tiers (Render, Railway, Fly.io, Netlify/Vercel for Angular)
- Monitoring: Application Insights or Grafana, uptime checks

■ APPLY (BUILD IT)

1. Write a `Dockerfile` for the API and for Angular (nginx serving the build)
2. Write `docker-compose.yml` so anyone can run the whole system with `docker compose up`
3. GitHub Actions: `ci.yml` (restore, build, test backend + frontend on every pull request) and `cd.yml` (build and push Docker images, deploy on merge to `main`)
4. Deploy the API, database, Redis and Angular app; connect a custom domain if possible
5. Set production environment variables and HTTPS
6. **Add the production URL to Google's Authorized JavaScript origins** and use the production Client ID

■ CHECKLIST (finish before moving on)

- ☐ The live URL works, and a public demo account is available
- ☐ Google Sign-In works on the live site (not only on localhost)
- ☐ Every push to `main` deploys automatically
- ☐ `docker compose up` runs the whole stack locally

Stage 15: Polish and CV Packaging

■ APPLY (BUILD IT)

1. **README.md** (recruiters read this first): title, one-paragraph description, live demo link, screenshots/GIFs, feature list with a ★ next to SignalR, Redis, Google Sign-In, payments and CI/CD
2. Add the architecture diagram and ERD, the tech stack table, and 'How to run locally' (`docker compose up`)
3. Add test coverage and CI badges, key technical decisions (why Clean Architecture, why Redis for the basket, why SignalR) and the performance results
4. Record a 2-3 minute demo video (customer flow, Google login, admin flow, real-time notification)
5. Provide demo accounts (`admin@demo.com`, `customer@demo.com`) and a test payment card

6. Clean the code: consistent naming, no dead code, meaningful commits, a few merged pull requests
7. Pin the repository on your GitHub profile and add the link to your CV and LinkedIn

Example CV bullets (adapt them to what you really built)

Oranos Shop: Full-Stack E-Commerce Platform | ASP.NET Core, Angular, SQL Server, Redis, SignalR, Docker

- Designed and built a full-stack e-commerce platform using **Clean Architecture, CQRS/MediatR** and the Repository and Specification patterns with ASP.NET Core Web API and Angular.
- Implemented **JWT authentication with refresh-token rotation**, role/policy-based authorization, ASP.NET Core Identity, and **Google Sign-In (OAuth 2.0 / OpenID Connect)** with server-side ID-token validation and account linking.
- Built **real-time order tracking, a live admin feed and customer chat with SignalR**, backed by a Redis backplane.
- Integrated **Stripe/Paymob payments with idempotent webhooks**, transactional order processing and concurrency-safe stock management.
- Reduced product-listing response time by X% using Redis caching, query optimization and indexing.
- Wrote unit, integration (Testcontainers) and E2E (Playwright) tests reaching X% coverage; set up **CI/CD with GitHub Actions** and containerized deployment with Docker.

Interview questions you should be able to answer after this project

- Why Clean Architecture? What does each layer depend on?
- Difference between Scoped, Transient and Singleton?
- How do you prevent N+1 queries in EF Core?
- How do JWT and refresh tokens work, and where do you store them safely?
- How does Google Sign-In work end to end? Why do you validate the ID token on the server, and what is the audience check for?
- What is the difference between OAuth 2.0 and OpenID Connect?
- How do you link a Google login to an existing email/password account safely?
- How does SignalR authenticate, and how do you scale it to multiple servers?
- How do you make a payment webhook idempotent?
- How do you handle two people buying the last item at the same time?
- What is the difference between an Observable and a Signal in Angular?
- Why lazy loading and **OnPush**?
- How does your CI/CD pipeline work?

Recommended Learning Resources

Topic	Resource
C# / .NET / EF Core / SignalR / Identity	learn.microsoft.com (official docs, the best source)
Google Sign-In	Google Identity Services documentation (developers.google.com/identity)
Angular	angular.dev (official tutorials)
RxJS	rxjs.dev
Roadmaps	roadmap.sh (ASP.NET Core, Angular, Docker, Redis paths)
Architecture	Search: 'Clean Architecture Jason Taylor template' and the Milan Jovanovic blog / YouTube
Full-course inspiration	Neil Cummings' 'Skinet' course (.NET + Angular e-commerce)
Testing	Testcontainers docs, Playwright docs
Docker / GitHub Actions	docs.docker.com, docs.github.com/actions

Golden Rules While Building

1. **Commit small and often**, with clear messages (`feat: add google login`). Recruiters read your history.
2. **Learn just enough, then build**. Do not watch 40 hours of tutorials before writing code.
3. **Write the README as you go**, not on the last day.
4. **Understand every line**. If you copy code from a tutorial or an AI, be able to explain it in an interview.
5. **Prefer finishing over adding**. A finished, deployed, tested project beats a bigger unfinished one.
6. **Keep a 'What I learned' file** for each stage. It becomes your interview cheat sheet.

Master Progress Tracker

- ☐ Stage 0: Prerequisites
- ☐ Stage 1: Planning and Design
- ☐ Stage 2: Backend Foundation
- ☐ Stage 3: Catalog API
- ☐ Stage 4: Authentication (JWT + Google Sign-In) ★
- ☐ Stage 5: Angular Foundation
- ☐ Stage 6: Storefront
- ☐ Stage 7: Basket (Redis)
- ☐ Stage 8: Checkout, Orders and Payments
- ☐ Stage 9: SignalR Real-Time ★
- ☐ Stage 10: Admin Dashboard
- ☐ Stage 11: Advanced Backend
- ☐ Stage 12: Testing

- ☐ Stage 13: Security and Performance
- ☐ Stage 14: DevOps and Deployment
- ☐ Stage 15: Polish and CV

Good luck! When you reach any stage, come back and ask: I can help with the code, explain a topic, review your architecture, or prepare interview questions for that stage.